

Movie Analytics Data Warehouse

Dimensional Modeling, Star Schema & OLAP Analysis

Student Name: Alessio Iacono | Student ID: 1870276

Course: Data Management - 25/26

Made with GAMMA

PROJECT OVERVIEW

Assignment Scope

Core Deliverables

- Integrate multiple heterogeneous data sources via ETL/ELT
- Define a Dimensional Fact Model (DFM)
- Build a star/snowflake schema on a relational database
- Execute OLAP analytical queries

Implementation Note

Built as an **ELT pipeline** using **dbt** and **DuckDB** instead of classic ETL/PostgreSQL pipeline.

Presentation Focus

Dimensional modeling design decisions and analytical querying patterns.



Made with GAMMA



PIPELINE ARCHITECTURE

ETL → ELT Migration

Declarative SQL transformations in dbt replace opaque visual NiFi flows with **version-controlled, testable, and documented** transformation logic, improving reproducibility and auditability.

Three-Layer Architecture

Three layers, virtual reconciled layer.
Source (`raw.*`) → Reconciled (`staging.*` + `intermediate.*`, materialized as dbt views) → DW (`marts.*`, materialized as tables).

Why virtual, not physical: at this data volume, materializing the reconciled layer buys nothing.

DuckDB recomputes the views cheaply, and a physical reconciled layer would mean an extra full write pass with no read-performance benefit anyone would notice.

Made with GAMMA

DATA SOURCES

Three Heterogeneous Sources

TMDB

Movie metadata + credits (cast/crew). **Nested JSON** arrays for genres and cast.

Box Office

Revenue figures (worldwide, domestic, foreign).
Flat **CSV** format.

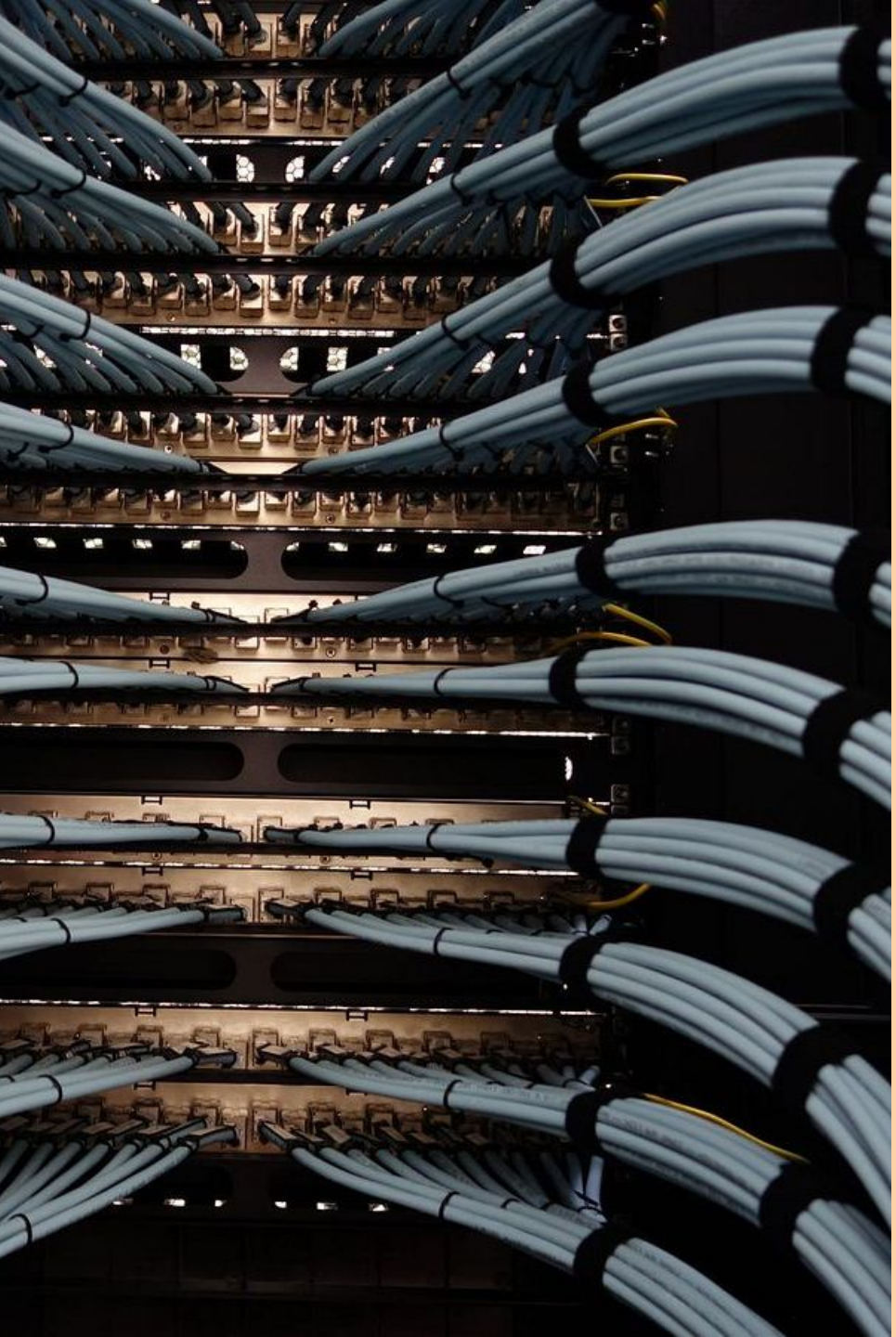
IMDb name.basics

Actor biographical data (birth/death year). Large **TSV** dump.

Key integration challenge: No shared identifiers across sources, different grains, and inconsistent formats require non-trivial matching logic.



Made with GAMMA



DATA INTEGRATION

Integration Highlights

1

Movie Matching (TMDB ↔️ Box Office)

Movies title matching on a 0–4 score: exact title+year, case-insensitive+year, ±1 year tolerance, substring match.

Deterministic tie-breaking via window function.

2

Person Matching (TMDB ↔️ IMDb)

Exact name matching with deterministic selection for name collisions.

3

Revenue Reconciliation

Box Office data takes priority when both sources agree on a movie, TMDB used as fallback.

4

Crew Role Whitelist

Only key production jobs retained via whitelist (*Director, Writer, Producer, Executive Producer, Director of Photography, Original Music Composer*).

Made with GAMMA

CONCEPCTUAL DESIGN

Dimensional Fact Model (DFM)

Two facts with different grain because different business question.

- `Movie Performance` answers "*how did a movie do*" (**1 row/movie**);
- `Movie Credit` answers "*who worked on what*" (**1 row/occurrence**).

Forcing both into one fact would mean picking a grain that's wrong for one of the two questions.

Conformance is asymmetric by design.

`Date` and `Movie` are true conformed dimensions, same surrogate key reused in both facts, enabling drill-across (e.g., "director's average profit" query).

`Person` and `Credit Role` are deliberately *local* to Movie Credit because they have no meaning in the Performance fact.

Factless Movie Credit fact.

There's no natural numeric measure for "occurred as a credit", the informative content is the occurrence itself.

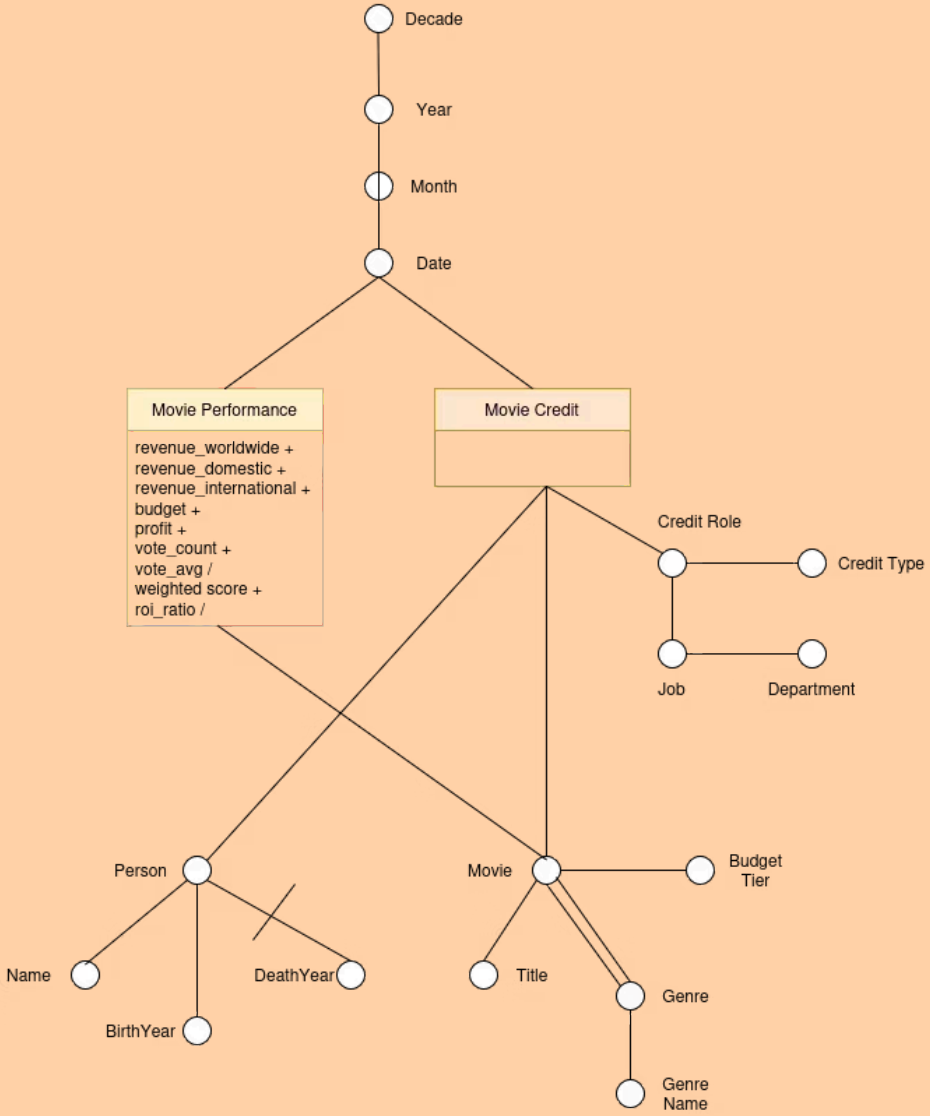
Per the DFM's empty-fact-schema pattern, we use `COUNT` at query time for coverage analysis rather than inventing a synthetic measure.

Additivity dictated the derived-measure design

`vote_avg` and `roi_ratio` are unit measures (ratios), *non-additive* along every dimension.

Rather than leaving this as a silent trap, `vote_count` and `weighted_score` are carried as explicit support measures so every analysis query recomputes the aggregate ratio from its components instead of naively averaging an average.

Made with GAMMA



Measure Dictionary			
Fact	**Measure**	**Definition/Formula**	**Additivity**
Movie Performance	*revenue_domestic*	Source value: domestic box-office revenue	Additive
Movie Performance	*revenue_worldwide*	Source value: worldwide box-office revenue	Additive
Movie Performance	*revenue_international*	*revenue_worldwide - revenue_domestic*	Additive
Movie Performance	*budget*	Source value: production budget	Additive
Movie Performance	*profit*	*revenue_worldwide - budget*	Additive
Movie Performance	*vote_count*	Source value: number of audience votes	Additive
Movie Performance	*vote_avg*	Source value: average audience rating	Non-additive; use AVG() or a weighted calculation
Movie Performance	*weighted_score*	*vote_count * vote_avg*	Additive
Movie Performance	*roi_ratio*	*profit / budget*	Non-additive; calculate from aggregated profit or budget

Derived Attributes		
Dimension	**Derived Attribute**	**Rule**
Movie	*budget_tier*	**Low Budget** when *budget < 10M*
Movie	*budget_tier*	**Mid Budget** when *10M <= budget <= 50M*
Movie	*budget_tier*	**Blockbuster** when *budget > 50M*

Made with GAMMA

LOGICAL DESIGN

Star Schema

Pure star schema, not snowflake. A cardinality decision.

Snowflaking pays off when secondary-to-primary tables cardinality is high, or when a hierarchy is shared across dimensions.

Neither applies here: job/department have a handful of values each, so dim_credit_role stays fully denormalized, normalizing it would add join cost for no space or integrity benefit.

Bridge table over push-down for the movie-genre M:N.

Push-down would replicate fct_movie_performance rows per genre, corrupting its grain (1 row/movie) and breaking the row-count-parity guarantee against dim_movie.

The bridge table keeps the fact grain intact and isolates the M:N resolution to a join, at the cost of one extra hop for genre-based queries, an accepted trade-off since genre filtering is common but not on the fact's critical path.

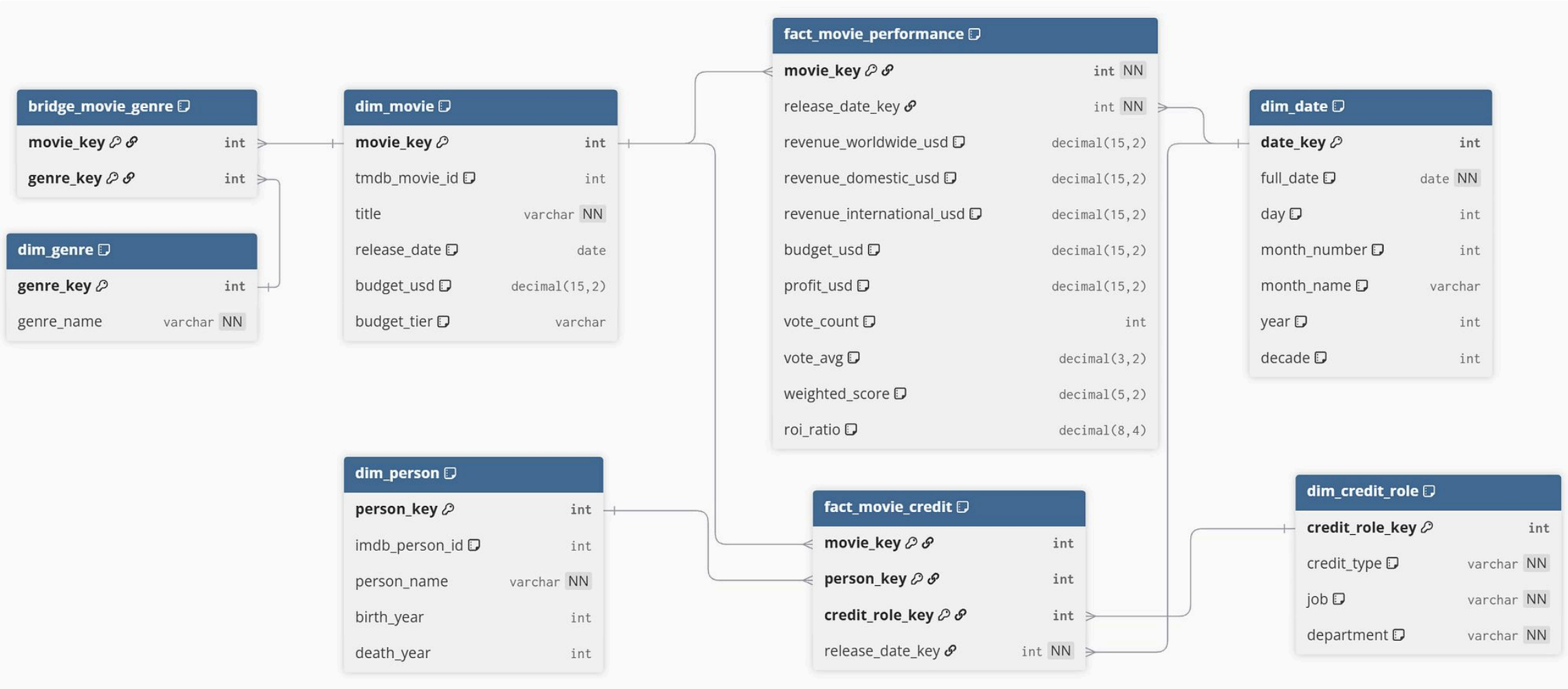
fct_movie_performance is a primary fact, not a view.

Despite containing derived measures (profit, roi_ratio), it sits at the finest available grain for its business process, nothing coarser is aggregated into it. Derived ≠ aggregated: these are computed from same-row columns at load time, not summed up from a lower-grain source.

budget_tier is a descriptive attribute, used as a hierarchy level anyway.

It's functionally determined by budget (not an independent dimension), which technically makes it descriptive rather than a "real" aggregation level but it's used for roll-up in the analyses regardless, a deliberate, pragmatic bend of the strict DFM distinction.

Made with GAMMA



Made with GAMMA

DATA QUALITY

Integrity Constraints

- **Functional dependencies:**
  - `movie_key` → `tmdb_movie_id`;
  - `date_key` → `full_date` → {`day`, `month`, `year`, `decade`} (derivation columns enforced by construction);
  - `credit_role_key` → {`credit_type`, `job`, `department`} *(uniqueness of this combination)*.
- **Domain constraints:**
  - `budget_tier` ∈ {Low Budget, Mid Budget, Blockbuster};
  - `credit_type` ∈ {CAST, CREW};
  - `job` restricted to a whitelist of key production roles *(added)*.
- **Nullability:** `revenue_worldwide_usd`, `revenue_domestic_usd`, `revenue_international_usd`, `budget_usd`, `profit_usd`, and `roi_ratio` may all be null when source coverage is incomplete. Intentionally untested for `not_null`.
- **Fact identification:**
  - `fct_movie_performance` by `movie_key` alone;
  - `fct_movie_credit` by (`movie_key`, `person_key`, `credit_role_key`).
- **Semantic rule:** `birth_year` < `death_year` when both populated.
- **Uniqueness:**
  - `genre_name` unique;
  - (`movie_key`, `genre_key`) unique in the bridge (no duplicate genre assignment).
- **Referential integrity:** every FK in both fact tables and the bridge table is enforced via `relationships` tests.

Additivity is a separate, non-testable convention: `vote_avg` *and* `roi_ratio` *must be recomputed from their support measures during aggregation, not tested at the row level.*

Made with GAMMA

DATA QUALITY

dbt Tests Replace Relational Constraints

dbt tests turn the integrity constraints from the previous slide into automated checks. Every `dbt build` or `dbt test` run re-verifies them, failing loudly if any constraint is violated.

Category	dbt test	Example
Structural	not_null, unique	*surrogate keys*: movie_key, date_key, person_key
Domain	accepted_values	budget_tier, credit_type
Composite Identity	dbt_utils.unique_combination_of_columns	fct_movie_credit (movie_key, person_key, credit_role_key)
Referential	relationship	any FK, e.g fct_movie_performance.movie_key → dim_movie
Custom business rule	dbt_utils.expression_is_true	worldwide = domestic + international

Made with GAMMA

ENGINE

Query Engine: ROLAP with Columnar Execution

G&R's ROLAP test: relational storage, SQL joins, no native cube structures. Explicit `dim_*`/`fct_*` tables, every analysis query a SQL join, no array storage, no MDX. Sparsity handling (a core ROLAP-vs-MOLAP concern in the theory) is moot here too: we only ever store tuples for events that actually occurred, never a full cross-product of dimension values.

DuckDB blurs the historical ROLAP/MOLAP performance line. Classic ROLAP (row-store, e.g. Postgres) trades relational flexibility for query cost. Joins are expensive, which is exactly why denormalization is ROLAP's standard performance fix.

DuckDB is **columnar and vectorized**: it keeps the relational/SQL model in full, but scans and aggregates at speeds historically associated with MOLAP's array-native access, without needing MOLAP's proprietary query language or its sparsity-management complexity.



Made with GAMMA

OLAP ANALYSIS

OLAP Query Examples

Q1 – ROI by Genre and Decade – roi_over_decade_and_genre.sql

Aggregates roi_ratio across genre and decade, filtering low-volume combinations and excluding early decades with unreliable financial data.

Demonstrates **Roll-Up** (aggregating roi_ratio from its support measures) and **Drill-Down** (decade → decade+genre) operators.

Q2 – Movies per Budget Tier by Year – movies_per_budget_size.sql

For each release year, counts distinct movies in each budget tier (Low/Mid/ Blockbuster) using DuckDB's native PIVOT clause, rotating budget_tier values into columns. Filtered to years where all three tiers are represented.

Demonstrates the **Pivot** OLAP operator.

Q3 – Director-Actor Profitability – best_director_actor_duo.sql

Self-join on fct_movie_credit to find frequent director/actor pairs, aggregating combined revenue and profit. Filtered to established collaborations (≥3 films together).

Demonstrates multi-fact join and self-join OLAP patterns, **Drill-across** operator.

Made with GAMMA

OLAP ANALYSIS

OLAP Operators Implemented

OLAP Operator	Description	Where
Roll-up	Increase aggregation, move up a hierarchy level (date → decade)	• revenue_by_decade.sql • movie_performance_by_budget_tier.sql
Drill-down	Add a dimension to reduce aggregation (decade → decade+genre)	• roi_over_decade_and_genre.sql
Slice	Fix one dimension to a single value	• sci-fi_revenue_by_decade.sql
Dice	Restrict on multiple dimensions at once	• total_revenue_scorsese_2010.sql • top_actors_movies_decade.sql • top_gross_top_rated_genre_decade.sql
Pivot	Rotate a dimensional attribute into columns	• movies_per_budget_size.sql
Drill-across	Join two fact tables via a conformed dimension	• best_director_actor_duo.sql • top_directors_by_avg_profit.sql

Made with GAMMA



Summary

Unified Schema

Three heterogeneous sources integrated into a single star schema via DFM-driven design.

OLAP Patterns

Queries demonstrate roll-up, drill-down, slice-and-dice, pivoting, and drill-across analytical operations.

Dimensional Design

Two facts at different grains, sharing conformed Date/Movie dimensions where it serves real drill-across needs.

Asymmetric conformance and a pure-star design driven by cardinality, not convention.

Made with GAMMA